

Challenge Técnico — IA Hub

Objetivo

El objetivo de este challenge es evaluar tu capacidad para diseñar e implementar una solución basada en **modelos de lenguaje (LLMs)** para extraer información estructurada desde texto libre. Buscamos evaluar no solo el resultado final, sino también tu forma de pensar, tu criterio técnico y tu capacidad de diseñar una solución mantenible.

En IA Hub trabajamos construyendo sistemas basados en modelos de lenguaje para automatizar procesos internos, estructurar información no organizada y asistir flujos de trabajo dentro de la Dirección de Producto y Tecnología. Este ejercicio simula un tipo de problema real que resolvemos en nuestro día a día.

El Challenge

Debés construir un sistema que reciba un texto libre y devuelva un **JSON estructurado**, siguiendo un esquema determinado. El objetivo es transformar texto no estructurado en datos confiables y utilizables.

Elegí una opción (A, B o C)

Podés elegir **una** de las siguientes opciones:

Opción A — Recetas

Dado un texto que describe una receta, generar un JSON estructurado con:

- nombre
- ingredientes (nombre, cantidad, unidad)
- pasos
- tiempo_estimado
- porciones
- dificultad
- tips

Output esperado:

```
{
  "input_original": "Tortilla de papas para 2 personas...",
  "output_estructurado": {
    "nombre": "Tortilla de papas",
    "ingredientes": [
      { "item": "huevos", "cantidad": 3, "unidad": "unidades" },
      { "item": "papas", "cantidad": 2, "unidad": "unidades" }
    ],
    "pasos": ["Pelar y cortar las papas", "Freírlas", "..."],
    "tiempo_estimado": "30 minutos",
    "porciones": 2,
    "dificultad": "media",
    "tips": ["usar sartén antiadherente"]
  },
  "warnings": []
}
```

Opción B — Feature Request

Dado un texto de solicitud de producto o feature, generar un JSON estructurado con:

- titulo
- descripcion
- usuario_objetivo
- problema
- solucion_propuesta
- prioridad (alta/media/baja)
- tags

Output esperado:

```
{
  "input_original": "Solicitud de dashboard de métricas...",
  "output_estructurado": {
    "titulo": "Dashboard de métricas en tiempo real",
    "descripcion": "Visualización de consumo de API para clientes enterprise",
    "usuario_objetivo": "Equipos de desarrollo enterprise",
    "problema": "No pueden monitorear consumo en tiempo real",
    "solucion_propuesta": "Dashboard con métricas, tendencias y alertas",
    "prioridad": "alta",
    "tags": ["analytics", "enterprise", "dashboard"]
  },
  "warnings": []
}
```

Opción C — Documento / Texto General 🏆 Recomendada

Dado un texto largo o semiestructurado (minutas, notas, resúmenes), generar un JSON estructurado con:

- titulo
- resumen
- entidades_clave
- conceptos_principales
- datos_relevantes
- acciones_sugeridas

Output esperado:

```
{
  "input_original": "Minuta de reunión de arquitectura...",
  "output_estructurado": {
    "titulo": "Definición de arquitectura para sistema de recomendaciones",
    "resumen": "Se acordó arquitectura híbrida usando vector DB y servicio FastAPI",
    "entidades_clave": ["API Gateway", "Pinecone", "FastAPI", "Redis"],
    "conceptos_principales": ["Filtrado colaborativo", "Embeddings", "Eventos Kafka"],
    "datos_relevantes": {
      "volumen_estimado": "10k requests/hora",
      "latencia_objetivo": "<200ms p95",
      "presupuesto": "$50k USD"
    },
    "acciones_sugeridas": [
      "Investigar vector DB (Juan)",
      "Preparar dataset (Ana)",
      "Definir métricas (María)"
    ]
  },
  "warnings": []
}
```

Requisitos Técnicos

Funcionalidad Core

1. **Input:** Recibir texto (TXT, o simplemente string)
2. **Procesamiento:** Enviar el contenido a un modelo LLM con prompt estructurado
3. **Extracción:** Obtener los campos definidos para tu opción elegida
4. **Validación:** Verificar coherencia de datos:
 - JSON válido y completo
 - Tipos de datos consistentes
 - Campos obligatorios presentes
5. **Output:** Generar JSON estructurado con campos extraídos + metadata (modelo usado, timestamp)
6. **Manejo de errores:** Casos de baja confianza o datos faltantes deben flaggearse en `warnings`

Modelos LLM de ejemplo

Categoría	Ejemplos
Privados	OpenAI (GPT-4o, GPT-4o-mini), Anthropic (Claude 3.5/3.7 Sonnet), Google (Gemini 2.5 Flash, 2.5 Pro)
Gratis/Freemium	GROQ (llama-3.2-90b, llama-3.1-70b, mixtral-8x7b)
Open Source	Llama 3.x, Mistral, Qwen2.5-VL, OCR Dolphin (especializado en documentos)

Elegí el modelo que prefieras. La elección queda a tu criterio - investigá opciones y justificá tu decisión en el `PROCESS.md`.

Recomendación: Si tenés GPU propia (RTX 3060+), probar local con Ollama es un plus. Si no, usar GROQ es la opción más accesible.

Datasets Públicos (Opcional)

Si querés probar con más datos, acá hay fuentes públicas:

Opción A — Recetas:

- **RecipeQA:** <https://hucvl.github.io/recipeqa/>
- **GitHub:** <https://github.com/josephrmartinez/recipe-dataset>
- **Dataset 200k+ recetas:** <https://paperswithcode.com/dataset/recipe1m>

Opción B — Feature Requests / Producto:

- **Kaggle:** <https://www.kaggle.com/datasets/ujjwalkarnel/product-feature-requests>
- **GitHub Issues:** <https://github.com/microsoft/vscode/issues>

Opción C — Documentos / Textos:

- **Kaggle:** <https://www.kaggle.com/datasets/shiyueh/english-meeting-summarization>
- **HuggingFace:** https://huggingface.co/datasets/cnn_dailymail

Requisitos Mínimos de Arquitectura

Se espera al menos una separación clara entre:

- Capa de interacción con el LLM
- Capa de validación/normalización del output
- Serialización del resultado final (JSON)

No buscamos sobreingeniería, pero sí claridad estructural y código mantenible.

Consideraciones de Producción (Opcional)

No es obligatorio implementarlo, pero valoramos que expliques (en README o PROCESS.md):

- Qué temperatura usarías y por qué
- Cómo manejarías fallos del proveedor (timeouts, respuestas inválidas, etc.)
- Cómo limitarías variabilidad en producción
- Cómo escalarías esta solución si recibiera miles de inputs diarios

Inputs de Prueba

Incluí **3 ejemplos de input** en tu repo, y mostrá sus outputs. Los inputs deben incluir:

- Un caso ideal
- Un caso ambiguo o incompleto
- Un caso ruidoso o difícil (texto largo, datos contradictorios, etc.)

Entregables

Tu entrega debe incluir:

1. Código Fuente

- Script principal de extracción
- Módulos de utilidad (validación, prompts, etc.)
- Configuración (API keys en .env, etc.)

2. README.md

- Descripción del proyecto
- Instrucciones de instalación y ejecución
- Ejemplo de uso con salida esperada
- Decisiones técnicas principales

3. MODEL_INFO.md (Obligatorio)

```
# Información del Modelo Usado

## Elección
- **Categoría:** [Privado / GROQ / Open Source]
- **Proveedor/Plataforma:** [OpenAI / GROQ / Ollama / etc.]
- **Modelo:** [gpt-4o / llama-3.2-90b / etc.]
- **Fecha de acceso:** [YYYY-MM-DD]

## Justificación
[Explicá en 2-3 oraciones por qué elegiste este modelo]

## Alternativas consideradas
[Mencioná otras opciones que evaluaste]
```

4. PROCESS.md (🔑 Fundamental)

```
# Proceso de Desarrollo

## Herramientas de IA Usadas
- [ ] GitHub Copilot
- [ ] Cursor
- [ ] ChatGPT
- [ ] Claude
- [ ] Otra: _____

## Mi Flujo de Trabajo

### 1. Planificación
[¿Cómo empezaste? ¿Diseñaste el schema primero?]

### 2. Prompts que me funcionaron
[Pegá prompts específicos que dieron buenos resultados]

### 3. Partes donde IA me ayudó
[¿Qué te generó la IA? ¿Qué revisaste/modificaste?]

### 4. Decisiones Técnicas
| Decisión | ¿Con ayuda de IA? | Justificación |
|-----|-----|-----|
| ... | Sí/No | ... |

### 5. Desafíos Encontrados
[¿Qué te costó más? ¿Cómo lo resolviste?]

### 6. Qué haría diferente
[Si tuvieras que hacerlo de nuevo]
```

5. Archivos de Prueba

- Al menos 3 ejemplos de input
- Un archivo `outputs.json` mostrando resultados

🕒 Tiempo Estimado

6–8 horas según el nivel de profundidad que elijas.

No buscamos una solución perfecta ni sobreingeniería, sino claridad de pensamiento y criterio técnico.

Criterios de Evaluación

Uso estratégico de IA (40%)

- Diseño claro del prompt
- Buen manejo de ambigüedades
- Validación del output del modelo
- Decisiones razonadas sobre temperatura y configuración

Calidad técnica (30%)

- Código claro y estructurado
- Separación de responsabilidades
- Manejo básico de errores
- Claridad en la arquitectura elegida

Documentación y proceso (30%)

- Claridad en el `PROCESS.md`
- Justificación de decisiones
- Reflexión sobre limitaciones

Malas señales

- Delegar completamente la lógica al LLM sin validación
 - No validar que el JSON sea consistente
 - Ignorar casos ambiguos o ruidosos
 - `PROCESS.md` vacío o genérico
-

Bonus (Opcional)

- Tests unitarios
 - Uso de funciones / tool calling
 - Reintentos ante fallos del modelo
 - Normalización inteligente de datos
 - Validación con JSON Schema
 - Dockerización
-

Entrega

Podés entregar el challenge como:

- Repo en GitHub (público o privado)
- ZIP con todo el contenido

Asegurate de incluir instrucciones claras para correrlo localmente.

Checklist Pre-Entrega

- ☐ El código corre sin errores en tu máquina
- ☐ Los 3 ejemplos de prueba producen output razonable
- ☐ README tiene instrucciones claras de instalación y uso
- ☐ `MODEL_INFO.md` está completo
- ☐ `PROCESS.md` está honestamente completado

- ☐ No hay API keys commiteadas
- ☐ Hay un archivo `.gitignore` apropiado

🎉 ¡Listo!
